

AMD ROCm Local Quantitative Investment Assistant

A private, auditable quant-investment agent running entirely on an AMD Radeon GPU with ROCm.

 gxinxing

 Qwen2.5-7B +
LoRA

 vLLM on ROCm

 gfx1100

 autoquant - local runtime

\$ > user: BTC 1h EMA20/50 金叉策略, 回测
+ 风险

\$ > intent: quantitative

\$ > dsl: validated (schema + semantic)

\$ > backtest: 180d synthetic · 156
trades

\$ > risk: APPROVE - drawdown 3.6%

\$ > report: auditable · source-aware

"The model proposes; deterministic code verifies; an independent Risk Agent has veto authority."

Ideas can't be trusted until they are measured.

Market research is full of natural-language ideas that cannot be trusted until grounded, structured, measured and risk-checked.

01

Ideas aren't executable

A verbal trading idea is untrustworthy until it is a validated, backtestable strategy.

02

Data & inference must stay local

Portfolio and research data cannot leave the private environment; no hosted model APIs.

03

No single layer of truth

An unchecked LLM output is not evidence — it needs deterministic verification and an independent veto.

Separation of concerns

The LLM proposes candidates. Deterministic code canonicalizes and validates them.
An independent risk layer can veto.

✦ PROPOSE

Language model

Qwen2.5-7B + LoRA converts natural language into a structured strategy DSL.

✓ VERIFY

Deterministic code

Canonicalization, schema + semantic validation, transpilation to executable engines.

🔒 VETO

Independent Risk Agent

Backtest sanity, drawdown / Sharpe / alpha gates, source-aware audit report.

Architecture



General questions → grounded assistant answer



AMD ROCm INFERENCE STACK

Qwen2.5-7B · FP16 LoRA · vLLM OpenAI-compatible endpoint · AMD Radeon gfx1100 (48 GiB) · ROCm 7.2.1

Interfaces: Open WebUI (chat) · Dify 6-node workflow · Gradio (optional) · Python Agent runtime

Required agent capabilities, with code evidence

1

Reasoning

ReAct Thought / Action / Observation loop

`src/agent/core.py`

2

Planning

Intent routing and dynamic tool sequencing

`src/agent/orchestrator.py`

3

Tool use

Knowledge, indicators, validation, backtest, report

`src/agent/tools.py`

4

Memory

Working, episodic, semantic tiers + preference extraction

`src/agent/memory.py`

5

Task execution

DSL, backtest, walk-forward, risk verdict

`src/dsl` · `src/backtest`

6

Self-healing

Allow-listed diagnose → repair → verify → rollback

`scripts/graph_engine.py`

Natural language → validated executable strategy

Never trusted as raw text — compiled through deterministic stages.



`{ }` EXAMPLE GENERATED DSL

```
strategy:
  name: EMA Trend_BTC
  indicators:
    - {name: ema_fast, type: EMA, params: {period: 20}}
    - {name: ema_slow, type: EMA, params: {period: 50}}
  entry: {long: "ema_fast > ema_slow"}
  risk: {stop_loss: -0.02, stake_amount: 0.1}
```


Optimization, measured

gfx1100

AMD Radeon GPU · 48 GiB
VRAM · ROCm 7.2.1

615 s

LoRA training · 400 samples · 39
steps

16.21 GB

peak training VRAM (FP16 LoRA)

98.1%

token accuracy · loss 0.2848

8.2 s

avg vLLM latency · 32K served
context

100%

ROCm-native local inference · no
hosted API

VERIFIED

Results

Deterministic synthetic data for full reproducibility — system behavior, not investment advice.

285

tests passed

24/24

CN-market evaluation

98.1%

token accuracy

6-node

Dify workflow · 3 demo cases

32K

served context tokens

4m23s

English-narrated demo video

R U N I T

One command, fully reproducible

No GPU, no model weights, no network, no real funds.

```
● ● ● pip install -r requirements.txt && python  
demos/run_track2_demo.py
```

On an AMD ROCm machine, the same script auto-uses the local vLLM model for DSL generation.



Live showcase (landing)

61a41b94d2884d3c8a0e5cdea2f8f218.bj6.agentos-app.net



Open WebUI live demo

minimize-orders-excel-saving.trycloudflare.com



Demo video (4m23s)

output/video/track2_demo_1080p_ava.mp4



Spec PDF

output/pdf/AMD_Quant_Assistant_Project_Specification.pdf

Track 2 submission alignment

1

Project Specification Document

Application scenarios, agent architecture, core capabilities, model + local deployment plan, AMD ROCm optimization



output/pdf/AMD_Quant...Project_Specification.pdf

2

Project Source Code + README

Full repo with env config, startup guide (manual + one-command demo) and dependency list



repository root · demos/run_track2_demo.py

3

Demo Video (3–5 min)

Real run on AMD Radeon GPU, CLI/GUI → final result, English narration



output/video/track2_demo_1080p_ava.mp4

4

Supplementary Material

PPT + poster highlighting creative scenarios and practical value



this deck · poster (PDF)

Thank you.

Track 2, gxinxing — AMD ROCm Local Quantitative Investment Assistant

github.com/gxinxing/Radeon-hackathon-2026-07



Local inference



Deterministic verification



Independent risk veto